

Local Polynomial Tensor Networks for Deep Neural Network Training

Exact tree representations, controlled changes of local model, and a teacher-student experimental protocol

Hierarchical feature learning project

1 October 2026

Abstract

We investigate whether successive local polynomial models can make the training of a smooth deep neural network interpretable without requiring a single tensor network to describe its entire trajectory. Replacing each activation by a degree- k Taylor polynomial produces a polynomial network. After adding a constant coordinate and unrolling repeated computations, this network has an exact representation as a weight-tied k -ary tree tensor network (TTN) evaluated on repeated copies of the input. We give the general construction and a complete depth-two, quadratic example. We then distinguish algebraic equivalence from dynamical approximation: the original parameterization must be retained, both prediction and parameter-Jacobian errors must be controlled, and re-expansion introduces measurable changes between successive local models. Conditional bounds describe within-window trajectory errors and their accumulation. Finally, we specify a teacher-student programme, beginning with staircase teachers, that tests whether these local models predict feature coefficients, singular subspaces, learning times, and intervention responses. Hierarchical SVD is applied to functions of actual input blocks; singular vectors of artificial repeated-input cuts are treated only as representation-dependent diagnostics.

Central claim. The tensor representation is exact for the polynomial surrogate. Its accuracy as a model of DNN training, its compressibility, and its usefulness for feature interpretation are three separate questions to be tested.

Reading guide. Sections 2-4 define and construct the model. Sections 5-7 establish its dynamical meaning and error accounting. Sections 8-11 specify feature observables and experiments. Appendix A gives derivative-error bounds; Appendix B specifies implementation checks.

1. Research question and scope

The project seeks an interpretable account of hierarchical feature learning: learning one representation may change how quickly another representation can be acquired. The existing teacher-student plan evaluates this through feature trajectories and causal interventions rather than loss agreement alone [P1]. Here we replace the requirement of a globally faithful tensor-network student by a weaker, local hypothesis:

Along DNN training, there are useful time windows on which a polynomial tensor model preserves the relevant gradient field and feature evolution. Successive models can be connected with an explicit error budget.

This note establishes algebraic identities and conditional approximation statements. It does not establish that low Taylor orders or small tensor ranks suffice for a realistic network, or that a hierarchical teacher forces a smooth MLP to learn hierarchically. Those are experimental hypotheses.

Two refinements are essential. First, expanding activations around fixed layerwise centres gives a polynomial in the input, but a short training interval does not by itself make the activation spread around those centres small. Expanding instead around each example's checkpoint preactivations gives a genuinely local-in-

parameters approximation, but its coefficients depend on the example. Second, the tree created by duplicating computations is not the tree of independent input blocks used to define task features. These distinctions determine both the theorem statements and the experiment design.

1.1 Relation to existing work

Layerwise activation Taylorization is used by NN2Poly to represent trained feed-forward networks as polynomials [R1]. Taylorized Training instead expands the whole predictor in parameter displacement, principally around initialization [R2]. The neural tangent hierarchy gives another route to controlled higher-order dynamical approximations under width and data assumptions [R3]. A 2026 preprint studies local optimization approximations at intermediate pretraining checkpoints [R4]; such evidence motivates local modelling but is not a result about TTN feature dynamics.

The tensor construction extends the project's tied repeated-input lifting for residual bilinear networks [P2, Sections 3 and 7]. Arithmetic-circuit/tensor-factorization correspondences [R5] and hierarchical SVD [R6] provide related algebraic tools. The feature-ODE paper [P5] supplies a motivating polynomial proxy, while explicitly not claiming a rigorous approximation chain from its MLP to that proxy. The present objective is to supply a controlled local comparison, not to identify that proxy with the construction below.

2. Smooth DNNs and activation Taylorization

2.1 Architecture, parameters, and loss

Let L denote the number of nonlinear hidden layers, independently of the depth of a teacher hierarchy. Let $d_0 = d$ and $d_1, \dots, d_L$ be the layer widths. For $x \in \mathbb{R}^d$, define

$$\begin{aligned} h_0(\theta, x) &= x, \\ z_\ell(\theta, x) &= W_\ell h_{\ell-1}(\theta, x) + b_\ell, \\ h_\ell(\theta, x) &= \sigma_\ell(z_\ell(\theta, x)), \quad 1 \leq \ell \leq L, \\ f_\theta(x) &= c^\top h_L(\theta, x) + b_{\text{out}}. \end{aligned}$$

Here $W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$, $b_\ell \in \mathbb{R}^{d_\ell}$, $c \in \mathbb{R}^{d_L}$, and θ contains all trainable entries. Each σ_ℓ acts coordinatewise, with scalar components of class C^{k+1} on the relevant preactivation intervals. The first experiments exclude dropout, normalization, and attention. An affine residual skip can be added without changing the polynomial argument.

For a data law ν on (x, y) , use

$$\mathcal{L}(\theta) = \frac{1}{2} \mathbb{E}_\nu[(f_\theta(x) - y)^2], \quad \dot{\theta} = -\nabla_\theta \mathcal{L}(\theta).$$

The same notation covers a fixed empirical distribution on a finite dataset. Population statements require the displayed expectations and derivative envelopes to be integrable; smoothness alone does not ensure this for unbounded inputs. Parameters use the Euclidean norm, equivalently the combined Frobenius norm on weight matrices and Euclidean norm on vectors.

2.2 A local model on window j

Choose checkpoint times $t_0 < t_1 < \dots$ and write $\theta_j = \theta(t_j)$. On window j , select centres $m_{\ell,j,i}$ and define

$$p_{\ell,j,i}(z) = \sum_{r=0}^k \alpha_{\ell,j,i,r} (z - m_{\ell,j,i})^r, \quad \alpha_{\ell,j,i,r} = \frac{\sigma_{\ell,i}^{(r)}(m_{\ell,j,i})}{r!}.$$

The centres and coefficients are **frozen during the window**. The weights and biases remain trainable. Replace each activation by its polynomial:

$$\begin{aligned}
\widehat{h}_{0,j}(\theta, x) &= x, \\
\widehat{z}_{\ell,j}(\theta, x) &= W_{\ell} \widehat{h}_{\ell-1,j}(\theta, x) + b_{\ell}, \\
\widehat{h}_{\ell,j}(\theta, x) &= p_{\ell,j}(\widehat{z}_{\ell,j}(\theta, x)), \\
\widehat{f}_j(\theta, x) &= c^{\top} \widehat{h}_{L,j}(\theta, x) + b_{\text{out}}.
\end{aligned}$$

All evaluations in this recurrence use the surrogate's own hidden states. Substituting the true network's hidden states at every layer would not define an autonomous surrogate forward pass.

For a scalar preactivation, Taylor's theorem gives

$$|\sigma(z) - p_j(z)| \leq \frac{M_{k+1}}{(k+1)!} |z - m_j|^{k+1}, \quad |\sigma'(z) - p'_j(z)| \leq \frac{M_{k+1}}{k!} |z - m_j|^k,$$

where M_{k+1} bounds $|\sigma^{(k+1)}|$ along the segment joining m_j and z . Bounds must cover the surrogate preactivations as well as the original ones.

2.3 Two different notions of locality

Construction A: fixed centres, polynomial in the input. Choose $m_{\ell,j}$ independently of x , for example the per-neuron mean of checkpoint preactivations on a calibration set. Then $\widehat{f}_j(\theta, \cdot)$ is a single polynomial with fixed input-independent tensor cores at each parameter value. This is the primary construction in Sections 3-4. Its error at θ_j is generally nonzero. Calibration means are a specified heuristic, not a theorem of optimal approximation.

Construction B: example-centred checkpoint expansion. For each x , set

$$m_{\ell,j}(x) = z_{\ell}(\theta_j, x).$$

Freeze these reference functions throughout the window. For a finite dataset, cache one centre vector per layer and example. For new examples, evaluate the frozen checkpoint network to obtain their centres. Then

$$\widehat{f}_j(\theta_j, x) = f_{\theta_j}(x), \quad D_{\theta}^r \widehat{f}_j(\theta_j, x) = D_{\theta}^r f_{\theta_j}(x) \quad (0 \leq r \leq k).$$

This equality of derivatives follows by induction through the layers: each activation replacement agrees to order k at its checkpoint preactivation, and smooth composition preserves that agreement. On a compact parameter neighbourhood with the required uniform or integrable derivative bounds,

$$\|f_{\theta_j+\Delta} - \widehat{f}_j(\theta_j + \Delta)\|_{L^2} = O(\|\Delta\|^{k+1}),$$

$$\|\nabla_{\theta} f_{\theta_j+\Delta} - \nabla_{\theta} \widehat{f}_j(\theta_j + \Delta)\|_{L^2} = O(\|\Delta\|^k).$$

Construction B has an example-conditioned tensor representation, but not generally one fixed-core polynomial in x . Its stored coefficients can retain complex nonlinear input dependence. It is therefore a useful temporal-approximation control, not evidence that the DNN has become a simple polynomial function of the input.

Important distinction. Layerwise degree k is not a degree- k Taylor expansion of the full predictor in θ . Composing the polynomial layers retains higher cross-layer terms. In particular, layerwise $k = 1$ is not the frozen-Jacobian/NTK model. For fixed centres, the maximum input degree is k^L , not k .

3. Exact tensor-network representation of the polynomial model

In this section use Construction A. Fix the window and suppress its index where harmless. The statements are algebraic and hold at every parameter value; approximation to the DNN is addressed later.

3.1 Polynomial tensors for one layer

Collect powers of the input state h :

$$P_\ell(h) = p_{\ell,j}(W_\ell h + b_\ell) = \sum_{r=0}^k A_\ell^{(r)}[h^{\otimes r}].$$

The symmetric r -linear map $A_\ell^{(r)}$ has one output index of size d_ℓ and r input indices of size $d_{\ell-1}$. The $r = 0$ tensor is a vector. If $w_{\ell,i}^\top$ is row i of W_ℓ and $q_{\ell,i} = b_{\ell,i} - m_{\ell,j,i}$, its explicit entries are

$$A_{\ell,i}^{(r)} = \left[\sum_{s=r}^k \frac{\sigma_{\ell,i}^{(s)}(m_{\ell,j,i})}{r!(s-r)!} q_{\ell,i}^{s-r} \right] w_{\ell,i}^{\otimes r}.$$

The formula follows from the binomial expansion of $(w_{\ell,i}^\top h + q_{\ell,i})^s$. It also shows that these cores are constrained: different polynomial orders reuse the same weight row. They are not independent dense trainable tensors.

3.2 Homogenization

Write $\bar{H}_\ell = \mathbb{R} \oplus \mathbb{R}^{d_\ell}$ and $\bar{h} = (1, h)$. Define the symmetric k -linear map

$$G_\ell : \bar{H}_{\ell-1}^{\otimes k} \longrightarrow \bar{H}_\ell$$

as follows. For $u_s = (a_s, v_s)$, $1 \leq s \leq k$, set

$$(G_\ell(u_1, \dots, u_k))_0 = \prod_{s=1}^k a_s,$$

$$(G_\ell(u_1, \dots, u_k))_{1:d_\ell} = \sum_{r=0}^k \binom{k}{r}^{-1} \sum_{\substack{S \subseteq \{1, \dots, k\} \\ |S|=r}} A_\ell^{(r)}[v_s : s \in S] \prod_{s \notin S} a_s.$$

An empty multilinear argument means $A_\ell^{(0)}$. Substituting $u_1 = \dots = u_k = (1, h)$ gives

$$\boxed{G_\ell(\bar{h}, \dots, \bar{h}) = (1, P_\ell(h)).}$$

The factors $\binom{k}{r}^{-1}$ prevent overcounting lower-degree terms. In coordinates, an entry with r nonzero input indices equals the corresponding entry of $A_\ell^{(r)}$ divided by $\binom{k}{r}$. The constant output has only the entry $(G_\ell)_{0\dots 0}^0 = 1$.

3.3 Unrolling gives a tree

Proposition 1 (exact tied-tree representation). For $k \geq 2$, the depth- L polynomial network has a representation by a rooted k -ary TTN with $N = k^L$ formal input leaves. All nodes at height ℓ use the same core G_ℓ . All leaves are evaluated at $\bar{x} = (1, x)$. With $\bar{c} = (b_{\text{out}}, c)$,

$$\widehat{f}_j(\theta, x) = \langle T_j(\theta), \bar{x}^{\otimes N} \rangle,$$

where T_j is the tensor obtained by contracting the tree cores and $\bar{c}$. Every bond carrying a height- ℓ hidden state has dimension at most $d_\ell + 1$.

Proof. Each homogenized layer takes k copies of the preceding augmented state. Recursively substitute the preceding layer into each occurrence. After L substitutions there are k^L leaves; the diagonal identity above proves equality at every node. The edge dimensions are the augmented hidden dimensions. The maximum polynomial degree follows by the same induction. For $k = 1$ the tree degenerates to a chain. $\square$

The network is a **tied TTN evaluated on repeated inputs**. Calling this a diagonal input restriction does not mean that its cores or edge matricizations are diagonal matrices. Ties and the algebraic relations among core entries are part of the representation.

A tree describes the unrolled multilinear expression, not necessarily the most efficient evaluation graph. The original sequential polynomial recurrence reuses identical subcomputations and need not perform exponentially many operations. Nor is there a fixed linear tensor that sends every vector h to $h \otimes h$: the construction duplicates the computation and supplies repeated formal arguments, as in [P2].

For $k > 2$, each higher-arity core can be decomposed into binary cores. An exact binary refinement always exists in finite dimensions, but its new bond dimensions depend on core matricization ranks; they are not automatically the original neural widths.

4. Worked example: two nonlinear layers and quadratic Taylorization

Consider

$$h_1 = \sigma_1(W_1x + b_1), \quad h_2 = \sigma_2(W_2h_1 + b_2), \quad f_\theta(x) = c^\top h_2 + b_{\text{out}}.$$

For each layer, let m_ℓ be fixed during the window, and define

$$s_\ell = \sigma_\ell(m_\ell), \quad D_\ell = D\sigma_\ell(m_\ell), \quad H_\ell = D^2\sigma_\ell(m_\ell), \quad q_\ell = b_\ell - m_\ell.$$

For coordinatewise activations, $(H_\ell[u, v])_i = \sigma''_{\ell,i}(m_{\ell,i})u_i v_i$. The quadratic layer is

$$P_\ell(h) = a_\ell + A_\ell h + B_\ell(h, h),$$

with

$$\begin{aligned} a_\ell &= s_\ell + D_\ell q_\ell + \frac{1}{2}H_\ell[q_\ell, q_\ell], \\ A_\ell u &= D_\ell W_\ell u + H_\ell[q_\ell, W_\ell u], \\ B_\ell(u, v) &= \frac{1}{2}H_\ell[W_\ell u, W_\ell v]. \end{aligned}$$

Thus B_ℓ is symmetric in its two arguments. The factor $1/2$ is already included in B_ℓ .

4.1 The two cores and their contraction

Define

$$G_\ell((\alpha, u), (\beta, v)) = \left(\alpha\beta, a_\ell\alpha\beta + \frac{1}{2}A_\ell(\alpha v + \beta u) + B_\ell(u, v) \right).$$

The constant output slice has only $(G_\ell)_{00}^0 = 1$. For output coordinate $i \geq 1$, the matrix of the two input indices is

$$(G_\ell)^i = \begin{pmatrix} (a_\ell)_i & \frac{1}{2}(A_\ell)_{i,:} \\ \frac{1}{2}(A_\ell)_{i,:}^\top & (B_\ell)_{i,:,:} \end{pmatrix}.$$

The local predictor is exactly

$$\widehat{f}_j(\theta, x) = \bar{c}^\top G_2(G_1(\bar{x}, \bar{x}), G_1(\bar{x}, \bar{x})).$$

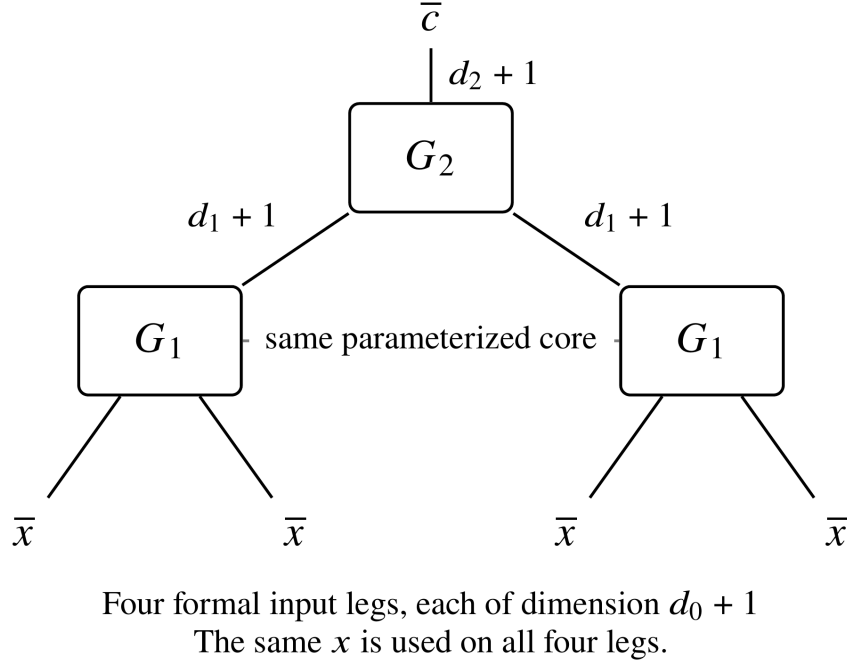


Figure 1: Binary TTN for the depth-two quadratic surrogate. The two lower cores are occurrences of the same parameterized core; all four formal leaves receive the same augmented input.

To expose the coefficient tensor, temporarily supply four separate formal arguments. Its entries are

$$T_{i_1 i_2 i_3 i_4} = \sum_{r,a,b} \bar{c}_r (G_2)_{ab}^r (G_1)_{i_1 i_2}^a (G_1)_{i_3 i_4}^b, \quad 0 \leq i_s \leq d_0.$$

The hidden indices a, b have size $d_1 + 1$ and r has size $d_2 + 1$. Contracting with $\bar{x}^{(1)}, \dots, \bar{x}^{(4)}$ defines a multilinear extension F_j . The original surrogate uses only

$$\widehat{f}_j(\theta, x) = F_j(\theta; x, x, x, x).$$

4.2 Why the output is quartic, not quadratic

Put $u = A_1 x$ and $v = B_1(x, x)$. The homogeneous input-degree contributions to the second hidden layer are

$$\begin{aligned} \text{degree 0 : } & a_2 + A_2 a_1 + B_2(a_1, a_1), \\ \text{degree 1 : } & A_2 u + 2B_2(a_1, u), \\ \text{degree 2 : } & A_2 v + 2B_2(a_1, v) + B_2(u, u), \\ \text{degree 3 : } & 2B_2(u, v), \\ \text{degree 4 : } & B_2(v, v). \end{aligned}$$

Multiply these vectors by $c^\top$ and add b_{out} to the constant term. Truncating the final predictor to total degree two would discard the last two lines and define a different approximation.

As a scalar check, take $\sigma(z) = z/(1 + e^{-z})$, both weights equal to one, zero biases, centres zero, and readout one. Its order-two activation polynomial is $p(z) = z/2 + z^2/4$. Therefore

$$p(p(x)) = \frac{x}{4} + \frac{3x^2}{16} + \frac{x^3}{16} + \frac{x^4}{64}.$$

The scalar core is $G((\alpha, u), (\beta, v)) = (\alpha\beta, (\alpha v + \beta u)/4 + uv/4)$. This illustrates the TTN representation of the approximation; it is not an assertion that the quartic equals the original smooth network globally.

5. Which training dynamics does the tensor network represent?

The algebraic equality holds as a function of the original parameters θ , so it also preserves derivatives **when differentiated through the same parameterization**. Define

$$\widehat{\mathcal{L}}_j(\theta) = \frac{1}{2} \mathbb{E}_v [(\widehat{f}_j(\theta, x) - y)^2].$$

Train the surrogate with $\tilde{\theta} = -\nabla_{\theta} \widehat{\mathcal{L}}_j(\tilde{\theta})$. The tensor cores are recomputed from the current W_{ℓ}, b_{ℓ} and the frozen activation coefficients. They are not new unconstrained optimization variables.

For example, if the two occurrences of G_1 in Section 4 are temporarily untied, the actual shared-core derivative is their **sum**. For a layer parameter W_{ℓ} ,

$$\nabla_{W_{\ell}} \widehat{\mathcal{L}}_j = (D_{W_{\ell}} G_{\ell, j})^* \sum_{v: \text{height}(v)=\ell} \nabla_{G_v} \mathcal{L}_{\text{tree}}.$$

This is the chain rule through all occurrences. Averaging instead of summing, untying the cores, or learning fixed constant-coordinate entries changes the dynamics [P2, Section 3.2].

More generally, stack the unique cores and readout into $g = \Gamma_j(\theta)$ and write the tensor objective as $\ell(g)$. On a frozen chart,

$$\dot{g} = -D\Gamma_j(\theta) D\Gamma_j(\theta)^{\top} \nabla_g \ell(g).$$

Thus Euclidean gradient flow in neural parameters induces a generally non-Euclidean, possibly degenerate evolution in core space. It is not ordinary gradient flow on freely chosen tensor entries. The right-hand side need not close as a function of a small set of singular values. Finding such a closure is an additional theoretical task.

Experimental rule. The primary local proxy and the reference DNN start from identical parameters and use identical data, learning rate, and optimizer state. No time rescaling is allowed for this comparison. Separately trained global TTN baselines can be compared under the weaker conventions of [P1], but they answer a different question.

6. Error control within one training window

6.1 Prediction and Jacobian defects

At the same parameter value, define

$$\begin{aligned} \varepsilon_{0,j}(\theta) &= \|f_{\theta} - \widehat{f}_j(\theta)\|_{L^2(v_x)}, \\ \varepsilon_{1,j}(\theta) &= \|\nabla_{\theta} f_{\theta} - \nabla_{\theta} \widehat{f}_j(\theta)\|_{L^2(v_x)}, \\ R_y(\theta) &= \|f_{\theta} - y\|_{L^2(v)}, \\ B_j(\theta) &= \|\nabla_{\theta} \widehat{f}_j(\theta)\|_{L^2(v_x)}. \end{aligned}$$

Vector-valued L^2 norms use the squared Euclidean norm inside the expectation. These quantities can refer either to the training empirical measure or to a population law, but the measure must be stated. Evaluation-set agreement alone does not bound the training gradient.

Proposition 2 (loss-gradient defect). Whenever differentiation under the expectation is valid,

$$\|\nabla \mathcal{L}(\theta) - \nabla \widehat{\mathcal{L}}_j(\theta)\| \leq R_y(\theta)\varepsilon_{1,j}(\theta) + B_j(\theta)\varepsilon_{0,j}(\theta) =: \beta_j(\theta).$$

Proof. With $J = \nabla_\theta f$ and $\widehat{J} = \nabla_\theta \widehat{f}$, the gradient difference is

$$\mathbb{E}[(f - y)(J - \widehat{J}) + (f - \widehat{f})\widehat{J}].$$

Apply the triangle inequality and Cauchy-Schwarz to the two terms. $\square$

A small function error alone is insufficient: two close predictors can have different parameter derivatives. Appendix A propagates activation and activation-derivative remainders into ε_0 and ε_1 .

6.2 Gradient-flow comparison

Let $F = -\nabla \mathcal{L}$ and $\widehat{F}_j = -\nabla \widehat{\mathcal{L}}_j$. Assume both compared trajectories remain in a common parameter tube on which F is K_j -Lipschitz. Let

$$E(s) = \|\theta(t_j + s) - \widetilde{\theta}(t_j + s)\|, \quad 0 \leq s \leq h_j := t_{j+1} - t_j.$$

Suppose $\|F - \widehat{F}_j\| \leq \bar{\beta}_j$ on this tube. Then

$$E(s) \leq e^{K_j s} E(0) + \bar{\beta}_j \frac{e^{K_j s} - 1}{K_j}.$$

The fraction is s when $K_j = 0$. To see this, add and subtract $F(\widetilde{\theta})$, obtaining the differential inequality $D^+ E \leq K_j E + \bar{\beta}_j$, and integrate it. No convexity or global convergence is assumed. A tube-exit or blow-up event terminates the guarantee.

For Construction B, if both paths start at θ_j , the surrogate speed is bounded by V_j , and the derivative bounds of Section 2.3 hold uniformly, then the time-dependent defect is at most $C_j(V_j s)^k$. Consequently,

$$E(h_j) \leq \frac{C_j e^{K_j h_j} V_j^k}{k+1} h_j^{k+1}.$$

This is a conditional local error estimate, not a universal practical window length. Constants can grow with depth, widths, parameter norms, activation derivatives, and the data distribution.

6.3 Why fixed-centre Taylorization has an error floor

For Construction A, the relevant radius already includes the spread of $z_\ell(\theta_j, x)$ around $m_{\ell,j}$. Schematically, on a bounded input set,

$$\|\widehat{z}_{\ell,j}(\theta, x) - m_{\ell,j}\| \leq S_{\ell,j} + C_{\ell,j}\|\theta - \theta_j\| + \|W_\ell\| \|\widehat{h}_{\ell-1,j} - h_{\ell-1}\|,$$

where $S_{\ell,j} = \sup_x \|z_\ell(\theta_j, x) - m_{\ell,j}\|$. Shrinking the time window removes the parameter-drift contribution, not $S_{\ell,j}$. A nonzero gradient defect at the checkpoint generally gives an $O(h_j)$ trajectory error. The $O(h_j^{k+1})$ estimate must not be applied to this construction without additional small-spread assumptions.

Gaussian inputs have unbounded support. A finite calibration maximum is not a population bound. One must either work with a stated finite empirical measure, prove integrable remainder estimates, or add

explicit tail contributions such as $\mathbb{E}[|f - \widehat{f}|^2 \mathbf{1}_{x \notin \mathcal{K}}]$. An empirical audit estimates these errors but is not a deterministic population certificate.

6.4 Discrete optimization

For matched full-batch gradient descent with step size η_n ,

$$\theta_{n+1} = \theta_n - \eta_n \nabla \mathcal{L}(\theta_n), \quad \widetilde{\theta}_{n+1} = \widetilde{\theta}_n - \eta_n \nabla \widehat{\mathcal{L}}_j(\widetilde{\theta}_n),$$

$$E_{n+1} \leq (1 + \eta_n K_j) E_n + \eta_n \beta_j(\widetilde{\theta}_n).$$

This compares two discrete algorithms directly and adds no artificial discretization term. Comparing either to gradient flow requires a separate numerical-integration bound. For SGD, use identical minibatch indices in the paired runs and bound the realized minibatch defect. Momentum and Adam-like methods require an augmented state containing optimizer memory; resetting that memory at a chart boundary is not a pure re-expansion.

7. Connecting successive local models

There are three distinct operations: advancing the trainable parameters, expressing an existing polynomial around a new centre, and replacing that polynomial by a fresh Taylor approximation. Only the last operation introduces new activation derivatives.

7.1 Exact translation of a polynomial versus fresh Taylor coefficients

For one scalar activation let $m' = m + \delta$ and $\alpha_r = \sigma^{(r)}(m)/r!$. The old polynomial, expressed in powers of $z - m'$, has coefficients

$$\widetilde{\alpha}_r = \sum_{s=r}^k \binom{s}{r} \alpha_s \delta^{s-r}.$$

Fresh coefficients at m' satisfy

$$\boxed{\alpha'_r = \widetilde{\alpha}_r + \rho_r, \quad |\rho_r| \leq \frac{M_{k+1} |\delta|^{k+1-r}}{r!(k+1-r)!}.$$

This follows by Taylor-expanding $\sigma^{(r)}(m + \delta)$ through degree $k - r$. The bound is componentwise for a vector activation. For $k = 2$ it reads

$$\begin{aligned} \alpha'_0 &= \alpha_0 + \alpha_1 \delta + \alpha_2 \delta^2 + \rho_0, & |\rho_0| &\leq M_3 |\delta|^3 / 6, \\ \alpha'_1 &= \alpha_1 + 2\alpha_2 \delta + \rho_1, & |\rho_1| &\leq M_3 |\delta|^2 / 2, \\ \alpha'_2 &= \alpha_2 + \rho_2, & |\rho_2| &\leq M_3 |\delta| / 2. \end{aligned}$$

Merely translating the old coefficients changes neither its function nor its approximation error. Re-evaluating derivatives supplies the ρ_r corrections. In particular, the highest retained coefficient can change at first order in the centre displacement. The finite jet is not an autonomous state without a rule for the omitted derivatives.

7.2 Function and gradient jumps at a common parameter

Let $r_j(\theta, x) = f_\theta(x) - \widehat{f}_j(\theta, x)$. At a parameter θ where both charts are defined,

$$\widehat{f}_{j+1}(\theta, x) - \widehat{f}_j(\theta, x) = r_j(\theta, x) - r_{j+1}(\theta, x).$$

Therefore

$$\|\widehat{f}_{j+1}(\theta) - \widehat{f}_j(\theta)\|_{L^2} \leq \varepsilon_{0,j}(\theta) + \varepsilon_{0,j+1}(\theta),$$

$$\|\nabla \widehat{\mathcal{L}}_{j+1}(\theta) - \nabla \widehat{\mathcal{L}}_j(\theta)\| \leq \beta_j(\theta) + \beta_{j+1}(\theta).$$

These bounds compare the same underlying parameter, not two different endpoints. The underlying DNN does not jump. Its piecewise surrogate can jump because the approximation has changed. For Construction B rebuilt at its own anchor, the new function and Jacobian defects vanish there, so the function jump is exactly the old residual.

7.3 Checkpoint-reset forecasts and carried-parameter forecasts

Checkpoint-reset forecasting. Start each local forecast from the recorded true θ_j , evolve the surrogate without access to future reference parameters, and compare it with the withheld DNN continuation. At the next test checkpoint, start a new independent forecast. This measures local predictive horizons. Repeated resets do not establish a globally accurate autonomous surrogate trajectory.

Carried-parameter forecasting. Preserve the surrogate endpoint as the next initial state and rebuild centres using that predicted state, not the reference DNN's next parameters. The rebuild may evaluate the original smooth architecture at the predicted parameters; count those evaluations. This is not a tensor-only simulator unless those refresh operations have also been replaced by a closed model.

For carried parameters, set $E_j = \|\theta(t_j) - \widetilde{\theta}(t_j)\|$. If a chart transition additionally perturbs the parameter state by at most κ_j , then

$$E_{j+1} \leq e^{K_j h_j} E_j + \overline{\beta}_j \frac{e^{K_j h_j} - 1}{K_j} + \kappa_j.$$

A pure change of Taylor centres has $\kappa_j = 0$. Projection into a new reduced parameterization may have $\kappa_j > 0$. For a uniform K , iteration gives

$$E_J \leq e^{K(t_J - t_0)} E_0 + \sum_{j=0}^{J-1} e^{K(t_J - t_{j+1})} \left[\overline{\beta}_j \frac{e^{K h_j} - 1}{K} + \kappa_j \right].$$

Chart jumps in the reported predictor are not additional parameter jumps; they must not be counted twice. If a checkpoint reset also changes parameters from θ^- to θ^+ , its displayed prediction jump is bounded by the chart mismatch plus a local predictor Lipschitz constant times $\|\theta^+ - \theta^-\|$.

7.4 Compression is another approximation, not a free consequence

If a compressed predictor $\widehat{f}_j^c$ is used dynamically, measure both

$$\delta_{0,j} = \|\widehat{f}_j - \widehat{f}_j^c\|_{L^2}, \quad \delta_{1,j} = \|\nabla_{\theta} \widehat{f}_j - \nabla_{\theta} \widehat{f}_j^c\|_{L^2}.$$

Add these to the prediction and Jacobian budgets in Proposition 2. A snapshot SVD approximation controls δ_0 , not automatically δ_1 . SVD truncation may also break core ties or the original parameterization; rank changes can spoil differentiability. Begin with compression for measurement only. Promoting it to a reduced dynamical model requires a separately specified differentiable parameter map or a controlled projected vector field.

8. Extracting features: distinguish two trees

8.1 The unrolled computation tree is not a partition of the data

In Proposition 1, every formal leaf contains the whole vector x . A cut of that tree separates computational occurrences, not disjoint physical variables. The diagonal data law is the pushforward of v_x under $x \mapsto (x, \dots, x)$, not N independent draws. Consequently,

$$\mathbb{E} [\langle \Delta T, \bar{x}^{\otimes N} \rangle^2] = \text{vec}(\Delta T)^\top \mathcal{M}_N \text{vec}(\Delta T), \quad \mathcal{M}_N = \mathbb{E}[v_N(x)v_N(x)^\top],$$

where $v_N(x) = \text{vec}(\bar{x}^{\otimes N})$. Generally $\mathcal{M}_N$ is not a Kronecker product of independent-leaf second moments. It can be singular because different extensions agree on the diagonal. A basic bound is

$$\|\langle \Delta T, \bar{x}^{\otimes N} \rangle\|_{L^2} \leq \|\Delta T\|_F \sqrt{\mathbb{E}[(1 + \|x\|^2)^N]},$$

when that moment exists; the factor may be very large [P2, Section 7].

The prescribed symmetric layer polarization chooses a definite extension, but its artificial-cut spectra are not intrinsic feature strengths of the diagonal predictor. Full symmetrization can remove extension ambiguity at the polynomial level while increasing ranks. **Do not read the artificial tree's bond singular values as learned task features without an additional justification.**

8.2 The physical input tree and functional SVD

Instead, let the real data have independent blocks

$$x = (x^{(1)}, \dots, x^{(M)}), \quad v_x = \mu_1 \otimes \dots \otimes \mu_M.$$

Fix a binary dimension tree $\mathcal{D}$ on these M blocks before examining the results. A node $S \subseteq \{1, \dots, M\}$ defines the physical cut $x_S \mid x_{S^c}$. For any square-integrable scalar predictor g , its kernel across this cut has a Schmidt/SVD expansion

$$g(x_S, x_{S^c}) = \sum_{a \geq 1} s_{S,a}(g) \phi_{S,a}(x_S) \psi_{S,a}(x_{S^c}),$$

with left and right functions orthonormal in the corresponding marginal L^2 spaces. These are architecture-independent output features. They are invariant under internal reparameterizations that leave g unchanged. Repeated singular values define subspaces, not uniquely labelled vectors. This is the feature convention of the residual teacher-student programme [P3, Section 4].

For Construction A, choose an orthonormal polynomial basis $\{\Psi_{i,a}\}$ in each real block. Under Gaussian blocks, use normalized multivariate probabilists' Hermites,

$$\Psi_{i,\alpha}(x^{(i)}) = \prod_q \frac{\text{He}_{\alpha_q}(x_q^{(i)})}{\sqrt{\alpha_q!}}$$

for whitened coordinates, with covariance-adapted coordinates otherwise. Expand

$$\hat{f}_j(\theta, x) = \sum_{a_1, \dots, a_M} C_j(\theta)_{a_1 \dots a_M} \prod_{i=1}^M \Psi_{i,a_i}(x^{(i)}).$$

A basis allowing block degrees up to k^L contains this polynomial exactly; most entries can be zero because the total degree is also at most k^L . Matricizing C_j across $S \mid S^c$ yields the functional SVD. Hierarchical

SVD can represent or approximate C_j by a TTN on **actual input blocks**. Its ranks are not bounded by the hidden widths in Proposition 1, because regrouping repeated input coordinates changes the tensorization. This separates two tasks: the unrolled TTN proves representability of the computation; the physical-block TTN organizes interpretable function-space features. Neither conversion nor low-rank compression is automatically cheap.

8.3 Error bars on singular values and feature subspaces

At a fixed parameter or forecast time, suppose $\|f - g\|_{L^2(v_x)} \leq \epsilon$. Let M_f and M_g be the associated cut operators. Their Hilbert-Schmidt difference has norm ϵ , so

$$\|M_f - M_g\|_{\text{op}} \leq \epsilon, \quad |s_{S,a}(f) - s_{S,a}(g)| \leq \epsilon.$$

Subspace comparison also requires a spectral gap. For the top r left singular subspace of M_f , define

$$\Delta = s_{S,r}(f)^2 - s_{S,r+1}(f)^2 > 0, \quad \zeta = 2\|M_f\|_{\text{op}}\epsilon + \epsilon^2.$$

Since $\|M_g M_g^* - M_f M_f^*\|_{\text{op}} \leq \zeta$, a Davis-Kahan bound gives, when $\zeta < \Delta/2$,

$$\|P_{S,r}(f) - P_{S,r}(g)\|_{\text{op}} \leq \frac{2\zeta}{\Delta}.$$

This is a direct application of spectral-subspace perturbation theory [R7]. Apply the analogous bound to right subspaces. A feature rotation near a degeneracy can be large even when prediction error is small; report the gap and compare the entire near-degenerate cluster.

At rollout time, the relevant function error includes both model defect and parameter drift:

$$\|f_{\theta(t)} - \widehat{f}_j(\widetilde{\theta}(t))\|_{L^2} \leq L_f \|\theta(t) - \widetilde{\theta}(t)\| + \varepsilon_{0,j}(\widetilde{\theta}(t)).$$

Add compression and basis-truncation errors when present. Sampling error in estimated cut operators is another contribution, not covered by the deterministic Taylor theorem.

Standard hierarchical truncation in an orthonormal product basis has a squared error bounded by a sum of discarded squared singular values over the truncation nodes [R6]:

$$\|C - C_{\mathbf{r}}\|_F^2 \leq \sum_{S \in \mathcal{D} \setminus \{\text{root}\}} \sum_{a > r_S} s_{S,a}(C)^2.$$

Using all non-root nodes is a conservative convention. Orthonormality converts this coefficient error into population function error. This statement does not apply unmodified to a raw repeated-input coefficient tensor.

8.4 A practical SVD estimator without expanding the coefficient tensor

For each physical cut, draw independently n_L samples $u_a \sim \mu_S$ and n_R samples $v_b \sim \mu_{S^c}$. Evaluate every recombined input and form

$$M_g[a, b] = \frac{g(u_a, v_b)}{\sqrt{n_L n_R}}.$$

Use the same row and column samples for the teacher, DNN, and proxy. If $M_g = U \Sigma V^\top$, the empirical feature values are $\sqrt{n_L} U_{:,q}$ on the left and $\sqrt{n_R} V_{:,q}$ on the right. On this empirical product measure,

$$\|M_f - M_g\|_F^2 = \frac{1}{n_L n_R} \sum_{a,b} |f(u_a, v_b) - g(u_a, v_b)|^2.$$

The singular-value and gap-dependent subspace bounds therefore apply directly to these finite matrices. Population accuracy requires a separate sampling analysis. The $n_L n_R$ entries are not independent samples: they reuse rows and columns. Estimate variability using independent grids or row-and-column resampling, not an i.i.d. bootstrap of matrix entries.

Begin with $n_L = n_R = 64$ and check stability at 128. This method evaluates the polynomial recurrence, not the exponentially large unrolled tree. It also works for Construction B or the original smooth DNN, although those functions need not have a finite polynomial coefficient tensor. Applying it across a consistent dimension tree estimates hierarchical cut features; independent cutwise SVDs alone do not construct a globally compatible compressed TTN.

8.5 What can become an interpretable dynamical theory?

On a fixed chart of Construction A, use a fixed orthonormal basis large enough for every surrogate on the window, and vectorize its coefficients as $C(\theta)$. For population square loss, let C^* denote the projection of the target into that basis. Orthogonal target components contribute only a constant. With $J_C = D_\theta C$,

$$\dot{C} = -J_C(\theta)J_C(\theta)^\top(C - C^*).$$

The equation is exact for the surrogate's original-parameter gradient flow. For empirical training, include the empirical feature Gram matrix rather than assuming population orthonormality. Across re-expansion, coefficient changes in this fixed orthonormal basis equal the function changes of Section 7.2 in norm.

One-step tensor matching. There is also an explicit relation between successive local coefficient tensors. Keep a common orthonormal basis large enough for both charts, and take one true GD step $\theta^+ = \theta - \eta \nabla \mathcal{L}(\theta)$. If $\|D_\theta^2 C_j\| \leq M_C$ along that parameter segment, then

$$C_{j+1}(\theta^+) = C_j(\theta) - \eta D_\theta C_j(\theta) \nabla \widehat{\mathcal{L}}_j(\theta) + \Xi_j,$$

$$\begin{aligned} \|\Xi_j\|_F &\leq \eta \|D_\theta C_j(\theta)\|_{\text{op}} \beta_j(\theta) + \frac{1}{2} \eta^2 M_C \|\nabla \mathcal{L}(\theta)\|^2 \\ &\quad + \varepsilon_{0,j}(\theta^+) + \varepsilon_{0,j+1}(\theta^+). \end{aligned}$$

The terms account respectively for the gradient-field defect, finite-step variation of the coefficient map, and the change of local chart. The identity follows by first-order Taylor expansion of C_j in the original parameters and then adding the same-parameter chart jump. It connects tensor evolution without identifying a representation change with physical learning. Compression or basis projection would introduce further terms.

For a simple singular value of a cut matrix $M(t)$,

$$\dot{s}_a = u_a^\top \dot{M} v_a.$$

This provides a route from the local tensor model to predictions for feature amplification. It is not a closed scalar ODE: J_C , singular vectors, and their couplings generally evolve. The research goal is to identify a small subset of these quantities whose evolution is accurately predictable.

Finally, an output SVD identifies structure used by the predictor, not the location of a computation in a hidden layer. Pair it with held-out teacher-feature probes and interventions. Record pre- and post-re-expansion spectra at the **same** parameters so that a chart correction is not misclassified as newly learned structure.

9. Teacher-student experiments with hierarchical targets

The starting families and feature observables follow the existing DNN training plan [P1] and the staircase formulation [P4]. The new comparison is between each trained smooth DNN and its own extracted local proxies, rather than only between independently trained architectures.

9.1 Common data and the staircase teacher

Let $x^{(i)} \sim \mathcal{N}(0, I_b)$ independently for $i = 1, \dots, M$. Choose fixed unit vectors $u_i \in \mathbb{R}^b$ and put

$$z_i = u_i^\top x^{(i)}, \quad \varphi_m(x) = \prod_{i=1}^m z_i, \quad f^*(x) = \sum_{m=1}^M \sqrt{\lambda_m} \varphi_m(x).$$

Here $\lambda_m > 0$ and initially $\sum_m \lambda_m = 1$. Independence and centring give $\mathbb{E}[\varphi_m \varphi_n] = \mathbf{1}_{m=n}$, so $\text{Var}(f^*) = 1$. Generate noise-free labels initially. The teacher has nested computational features, but the DNN's acquisition order is not assumed.

For any predictor g_t , define the architecture-independent coefficients

$$a_m^g(t) = \frac{\mathbb{E}[g_t(x) \varphi_m(x)]}{\sqrt{\lambda_m}}.$$

The teacher has $a_m^{f^*} = 1$. If two predictors are within ϵ in L^2 , their normalized coefficient difference is at most $\epsilon/\sqrt{\lambda_m}$. For coefficients $c_m = \mathbb{E}[g \varphi_m]$, the exact population loss decomposition is

$$\frac{1}{2} \|g - f^*\|_{L^2}^2 = \frac{1}{2} \sum_m (c_m - \sqrt{\lambda_m})^2 + \frac{1}{2} \left\| g - \sum_m c_m \varphi_m \right\|_{L^2}^2.$$

Track the last term: fitting teacher components while creating substantial spurious output is not full success.

The matched staircase positive-control student is

$$g_W(x) = \sum_{m=1}^M \sqrt{\lambda_m} \prod_{i=1}^m W_i^\top x^{(i)}.$$

The shared W_i and fixed readout strengths are part of this parameterization. The project's gating results concern this specified model and regime [P4]; they are not transferred as assumptions to a generic TTN or smooth MLP.

9.2 A progressively harder teacher family

T1: wider, branching polynomial hierarchy. Replace the chain by a balanced tree. At each leaf set $h_i^* = U_i x^{(i)} \in \mathbb{R}^r$, and at an internal node use $h_v^* = B_v(h_{v_L}^*, h_{v_R}^*)$. Choose low-rank planted bilinear cores and allow a fixed sum of readouts from selected internal nodes. Compare $r = 1, 2, 4$. Store the teacher's full specification and evaluate its physical-cut SVD, rather than assuming planted hidden coordinates are orthogonal features.

T2: residual TTN with bypasses. Use

$$h_v^* = \rho(h_{v_L}^* + h_{v_R}^*) + \gamma B_v(h_{v_L}^*, h_{v_R}^*), \quad f^* = c^{*\top} h_{\text{root}}^*.$$

Sweep $\rho \in \{0, 0.25, 1\}$ with fixed $\gamma = 1$ initially. This changes available routes, not a guaranteed direction of causal dependence. Normalize the teacher output once per baseline condition using a separate calibration calculation, and store that constant. This family is the project's soft-hierarchy test [P2-P3].

T3: smooth planted compositional hierarchy. For scalar leaf features $\varphi_i = z_i$, assign each internal node a named feature

$$\varphi_v = \sigma(a_v \varphi_{v_L} + b_v \varphi_{v_R} + \gamma_v \varphi_{v_L} \varphi_{v_R} + d_v), \quad f^* = \sum_{v \in \mathcal{V}} \omega_v \varphi_v.$$

Choose and freeze nonzero coefficients, retain readouts at several depths, and standardize each teacher feature by fixed calibration mean and variance if needed. This preserves an explicit compositional graph while removing exact polynomiality. Next introduce a second route to a parent or reuse a lower feature in several parents. Record the resulting directed acyclic graph and retain physical-cut SVD as the common output-level observable. Computational hierarchy remains known; whether any student acquires it causally is still tested.

Only after these stages add anisotropic block covariance, label noise, SGD, adaptive optimizers, or normalization. Independence of real blocks can be retained when covariance within a block is anisotropic. Correlation across blocks requires changing the functional-SVD interpretation rather than silently using a product metric.

9.3 Students and local proxy controls

Train a smooth MLP and, next, a residual smooth MLP on the same training samples. Use SiLU as the initial activation; use tanh or an explicitly specified GELU implementation as a robustness check. Record the exact activation formula, since Taylor coefficients must match the implementation.

For each DNN checkpoint compare: the original DNN continuation; fixed-centre layerwise Taylor proxies for $k = 1, 2, 3, 4$; the example-centred proxy; and the full-predictor parameter-linearized model $f_{\theta_j} + \nabla_{\theta} f_{\theta_j}^T (\theta - \theta_j)$. A full parameter-quadratic Taylor model is an optional small-network control. These isolate layerwise composition from ordinary parameter linearization.

Separately train the matched staircase model and a quadratic-activation MLP as positive and algebraic bridge controls. They need not have parameters that correspond to those of the smooth DNN. Do not confuse agreement of an extracted proxy with success of an independently trained TTN student.

10. A concrete experiment sequence

The numerical choices below are proposed defaults, not derived optimal values. Increase complexity only after checking the earlier stage.

10.1 Initial configuration

Component	First configuration
Teacher	Staircase with $M = 3$, block dimension $b = 4$, $\lambda_m = 1/3$; repeat at $M = 4$.
DNN	$L = 2$ hidden layers of width 32, SiLU, scalar affine output. Later vary $L \in \{2, 3, 4\}$ and width 64.
Initialization	Independent Gaussian weights with standard deviation $s/\sqrt{d_{\text{in}}}$, zero biases, including the scaled random output head; start with $s = 1$, then test $s = 0.5$.
Data	4,096 training, 1,024 centre-calibration, 4,096 audit, and 8,192 final-test examples; all disjoint, with fixed seeds.
Optimization	Full-batch GD, no momentum, decay, noise, or normalization. Start at $\eta = 0.01$ and use the pilot stability procedure below.
Training record	4,400 updates; reference checkpoints at updates 0, 100, 500, 1,000, 2,000, and 4,000. Record ordinary metrics every 10 updates.

Component	First configuration
Local forecasts	Window lengths $H \in \{1, 5, 20, 100, 400\}$ updates and Taylor orders $k \in \{1, 2, 3, 4\}$. Centres remain frozen throughout each forecast.
Replication	Five paired teacher/initialization seeds initially. All proxy variants for a seed reuse the same checkpoint and data.
Feature diagnostics	Staircase coefficients throughout; crossed-grid SVD on the physical cuts with 64 rows and 64 columns, checked at 128.

Choose the learning rate on a separate pilot teacher from $\{0.001, 0.003, 0.01, 0.03\}$. Reject divergent choices and check that halving the step size approximately preserves the reference feature curves when plotted against $t = \eta n$. Freeze the choice before evaluating the main seeds. A fixed dataset is an empirical training problem; increasing its size and using fresh test samples are necessary checks before interpreting it as population training.

10.2 Experiment E0: verify the algebra and parameterization

Use a tiny depth-two model with widths $(d_0, d_1, d_2) = (3, 4, 2)$ in float64. Compare the sequential quadratic recurrence, the tied tree contraction, and the explicit order-four coefficient contraction. Compare both values and gradients with respect to the **original** weights, biases, and output head. A practical tolerance is 10^{-10} in relative prediction error and 10^{-8} in relative gradient error, with a stated absolute floor near zero.

Repeat with $\sigma(z) = z + z^2$. Its degree-two Taylor replacement is exact for every centre and parameter value, so there should be no Taylor error. A mismatch indicates a construction, normalization, or sharing bug. Deliberately untying occurrences or averaging their gradients should fail the parameter-gradient equivalence test. Use the scalar example of Section 4 as an independent coefficient check.

10.3 Experiment E1: separate temporal approximation from input spread

At each checkpoint, construct A from calibration means and B from the checkpoint’s per-example preactivations. Measure their prediction and gradient defects at the checkpoint before any update. Then perturb parameters along fixed unit directions with radii $10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}$, retaining only perturbations inside a valid smooth bounded region.

For B, test the $O(\|\Delta\|^{k+1})$ prediction and $O(\|\Delta\|^k)$ Jacobian upper-order behaviour. Vanishing leading coefficients can improve the observed order, and floating-point error eventually dominates. For A, explicitly report a nonzero initial defect and its dependence on activation spread. Do not subtract this floor and then report a temporal order as if the total error vanished.

At larger parameter counts, Jacobian-vector products along independent random directions can screen derivative mismatch. These are diagnostics unless accompanied by a justified estimator or bound. Always measure the full actual loss-gradient difference at audited states; it directly tests the vector field relevant to that training objective.

Decision. If A fails already at update zero while B succeeds, the problem is input-wide activation approximation, not the TTN identity. Possible next tests are higher order, multiple fixed input patches, or a derivative-aware distributional polynomial fit. A patch mixture is piecewise polynomial, not one global polynomial; a fitted polynomial is not automatically a Taylor model. Label these variants separately.

10.4 Experiment E2: blind local rollout prediction

At checkpoint j , clone the same parameters into the DNN continuation and each proxy. Train each for H updates on identical training data without loading future reference parameters. Evaluate at updates

0, 1, 5, 20, 100, 400 whenever they lie inside the window. Freeze both centre choice and stopping policy before the held-out continuation is inspected.

Report prediction error, loss difference, parameter error, loss-gradient error at common parameter points, and teacher-feature coefficient error. Distinguish the common-point defect $\widehat{f}_j(\theta) - f_\theta$ from the total rollout difference $\widehat{f}_j(\widetilde{\theta}) - f_\theta$.

As initial, explicit accuracy targets use normalized prediction RMS error at most 0.02, maximum normalized staircase-coefficient error at most 0.05, and relative loss-gradient defect at most 0.10 on audited states. Normalize prediction error by the teacher RMS. For the gradient denominator use $\max(\|\nabla \mathcal{L}\|, g_{\min})$, where $g_{\min} = 10^{-6} \max(1, \|\nabla \mathcal{L}(\theta_0)\|)$, and also report the unnormalized defect. These are experimental tolerances, not mathematical guarantees.

Plot the largest accepted horizon against checkpoint, Taylor order, activation spread, and gradient defect. A proxy that succeeds only for an infinitesimal interval may be formally correct but not useful for explaining feature acquisition. A matching loss curve with inaccurate feature coefficients is a failure of feature-level prediction.

10.5 Experiment E3: stitching and refresh costs

Compare three policies: one polynomial model frozen at the initial checkpoint; periodic checkpoint-reset forecasts; and carried-parameter forecasts with refresh intervals fixed using pilot results. For the latter, refresh centres from the predicted parameters and keep the optimizer state. Report endpoint error, per-window error growth, predictor and spectrum jumps at refresh, and the number of smooth-network evaluations needed for refresh and auditing.

A remainder-based adaptive policy may use valid interval derivative bounds and the radii from Section 6.3. A difference between order- k and order- $(k+1)$ proxies is only a heuristic error indicator unless its reliability is proved. Likewise, a rule that repeatedly queries the reference gradient is an audited/oracle-assisted policy, not an autonomous forecast. Keep these policies in separate result columns.

10.6 Experiment E4: feature extraction and compression

For the known physical dimension tree, compare teacher, DNN, and proxy singular values and top- r feature subspaces on identical crossed grids. Begin with $r = 1, 2, 4$ where the corresponding singular-value gaps are resolved. Use

$$A_{S,r}(g, f^*) = \frac{1}{r} \|U_{S,r}(g)^\top U_{S,r}(f^*)\|_F^2$$

on a shared empirical grid, and report the same DNN-proxy comparison. Also report singular amplitudes: high alignment in a nearly zero-amplitude subspace is not acquisition of a useful feature. Treat unresolved ranks and nearly degenerate clusters as such, rather than selecting a convenient basis.

In small polynomial cases, explicitly convert the surrogate into a physical-block coefficient tensor, apply hierarchical SVD at several ranks, and compare held-out function error with discarded singular mass. Otherwise use the crossed-grid features first; they are not a certificate of global TTN compression. Only after observational compression succeeds test its derivative error and dynamical validity.

Fit held-out linear probes from every DNN and proxy hidden layer to the teacher's named features. Use separate probe-fit and probe-test samples; choose regularization without the probe-test labels. These probes test whether output-level feature changes correspond to accessible internal computations. They are supporting measurements, not evidence of necessity on their own.

10.7 Experiment E5: causal tests and controls

The simplest intervention follows [P1]: replace λ_2 by $\lambda_2/4$ while keeping the other coefficients, teacher directions, data samples, initialization, and learning rate fixed. Do not renormalize the perturbed target

back to variance one; that would change every channel. Run paired training experiments and paired local forecasts. A finite-difference clock coupling is

$$\widehat{C}_{2 \rightarrow 3} = -\frac{\log \tau_3(\lambda_2/4) - \log \tau_3(\lambda_2)}{\log(1/4)}.$$

A positive value indicates a delay after weakening the proposed prerequisite. It is a task-strength intervention, not a direct intervention on an internal representation, so complement it with a feature-blocking test.

For a reproducible blocking test, fit a teacher-feature probe at one checkpoint and layer using calibration samples. Centre and whiten that layer’s activations on its supported covariance subspace. Let q be the unit direction most correlated with a declared teacher feature in these whitened coordinates. Freeze all calibration statistics and replace z by $(I - qq^\top)z$ before mapping back to hidden coordinates. This is a fixed affine intervention and can be inserted in both the DNN and polynomial proxy. Rebuild each local model for the intervened architecture, then compare continuation from the same parameters.

Measure the immediate forward shock separately from subsequent learning. Verify that the targeted probe signal was actually suppressed; residual routes may preserve it elsewhere. Use an identity/sham intervention and a random direction in the same supported subspace as controls. The protocol does not equate arbitrary probe directions with causally necessary features.

Use acquisition times $\tau_m(a) = \inf\{t : a_m(t) \geq a\}$ at $a \in \{0.1, 0.25, 0.5\}$, with a predeclared persistence requirement of five consecutive recorded checkpoints for noisy measurements. For local forecasts, define crossing time relative to the forecast start. If no crossing occurs before the horizon, report right-censoring, not an infinite time or an invented ratio. Small trajectory errors imply small hitting-time errors only for a separated, transverse crossing; flat plateaus require explicit uncertainty.

Finally, include degree-matched non-nested targets, reversed feature strengths, and an incorrect physical dimension tree. Disjoint-support targets may change the number of relevant directions even when ambient dimension and polynomial degrees match; report that confound rather than treating the control as perfectly matched. The wrong-tree control tests interpretability and compressibility, not the validity of the DNN’s forward pass.

11. Reproducible implementation and decision criteria

11.1 Minimal local-forecast algorithm

Inputs: teacher, data splits, initial DNN parameters, checkpoint schedule, Taylor orders, centre policies, forecast lengths, and optimizer.

Train the reference DNN and save parameters plus full optimizer state.

For each checkpoint j :

Keep the future reference continuation inaccessible to construction.

Record true checkpoint preactivations on calibration/training inputs.

For each centre policy and Taylor order k :

Compute centres and derivative coefficients; freeze and detach them.

Clone checkpoint parameters and optimizer state into the proxy.

Check zero-step prediction and gradient defects.

For each forecast length H :

Restart this forecast from the identical cloned checkpoint.

Run H proxy updates using its own hidden states and the same data.

Do not replace any proxy parameter by a future reference value.

Audit predictions, gradients, coefficients, and feature subspaces.

Separately evaluate old and refreshed charts at the same endpoint.

For carried-parameter tests:

Refresh from predicted parameters; do not reload reference checkpoints.

Aggregate paired-seed errors, horizons, feature curves, and interventions.

In an autodifferentiation implementation, centres and all $\sigma^{(r)}(m)$ must be detached from the live training graph. Otherwise differentiation through the changing Taylor centre defines a different update rule. Recomputing the tensors $A_\ell^{(r)}$ or G_ℓ from live weights must remain differentiable. Prefer the efficient polynomial recurrence for rollout; use explicit tensor contractions only for equivalence tests and small spectral analyses.

11.2 What to save

Store the teacher graph and all teacher weights, normalization constants, data split seeds or hashes, exact activation formulas, parameter initialization, learning-rate schedule, optimizer state, checkpoint indices, centre policy, frozen centres, derivative coefficients, and Taylor order. Each forecast record should identify whether it is checkpoint-reset, carried, or oracle-audited, and whether compression affected measurement or training.

Save absolute and normalized errors separately, actual gradient norms, input/preactivation ranges, tensor or functional singular values, resolved gaps, feature coefficients, and centre-refresh jumps. Keep the same feature basis or evaluation grid across adjacent checkpoints. Include wall time and memory measurements, but do not assume the algebraic TTN representation is a speedup.

For each scalar curve report paired differences across seeds, not only independent confidence bands. Use independent test examples for coefficient uncertainty and independent crossed grids for SVD stability. Separate calibration, audit-based model selection, and the final held-out evaluation. A five-seed first screen is exploratory; strengthen replication before claiming robust scaling or causal laws.

11.3 Success, failure, and the next theoretical step

Local dynamical fidelity: a proxy predicts a nontrivial continuation of the same DNN under the same optimization, without future-trajectory fitting. Improvement with Taylor order should survive fixed held-out configurations rather than only the settings used to choose centres.

Feature fidelity: teacher coefficients and resolved physical-cut subspaces agree within measured approximation and sampling error. A model passes this criterion only if chart-refresh artefacts are separated from continuous parameter-driven changes.

Mechanistic fidelity: prerequisite-strength changes and controlled feature blocking have comparable downstream effects in the DNN and proxy. Agreement in unperturbed feature ordering is not sufficient.

Reduction: a substantially smaller representation or set of dynamical variables preserves these predictions. If full-width polynomial simulation works but no useful reduction exists, the result is a controlled representation, not yet a tractable theory.

A small successful example motivates deriving a reduced evolution for selected coefficient modes, singular subspaces, and tangent-kernel couplings. A fixed-centre failure with example-centred success motivates a better input-distribution approximation. Accurate predictions but unstable feature spectra motivate better observables or gap-aware subspaces. Failure even of example-centred local forecasts at the predicted radius calls for checking derivative bounds, optimizer-state handling, and numerical implementation before changing the tensor hypothesis.

12. Main conclusions and testable hypotheses

The exact part of the programme is

$$\text{layerwise polynomial surrogate} = \text{constrained, tied } k\text{-ary TTN on repeated inputs.}$$

The dynamical bridge is conditional on a small loss-gradient defect, correct parameterization, and controlled transitions between local models. The interpretability bridge uses the actual input distribution and physical variable cuts; it does not follow from an arbitrary SVD of the unrolled computation tensor.

The proposed hypotheses are that useful low-order windows exist, that their physical-block feature structure is compressible, and that a small set of such features predicts hierarchical learning and its response to interventions. None is assumed by the representation theorem. The first milestone is therefore a depth-two SiLU student learning a three-stage staircase target, with verified tensor identities, measured prediction and gradient defects, blind local forecasts, and gap-aware feature comparisons. Only then should the programme progress to wider, residual, and smooth compositional teachers.

Appendix A. Propagating activation errors to parameter derivatives

This appendix gives one sufficient finite-dimensional error calculation. It is deliberately conservative. Work on a bounded parameter tube and input set, using Euclidean activation norms and operator norms of derivatives. Assume the required true, surrogate, and connecting preactivations lie in the stated regions.

For layer ℓ , let w_ℓ bound $\|W_\ell\|$, let s_ℓ bound $\|D\sigma_\ell\|$, and let $h_\ell^{(2)}$ be a Lipschitz constant for $D\sigma_\ell$. If every surrogate coordinate is within R_ℓ of its Taylor centre and M_ℓ bounds the scalar $(k+1)$ st derivatives, set

$$\epsilon_\ell = \sqrt{d_\ell} \frac{M_\ell R_\ell^{k+1}}{(k+1)!}, \quad \epsilon'_\ell = \frac{M_\ell R_\ell^k}{k!}.$$

The factor $\sqrt{d_\ell}$ converts coordinatewise value bounds to a vector norm; it is not needed for the operator norm of a diagonal derivative error. Let

$$e_\ell = \sup \|h_\ell - \widehat{h}_\ell\|, \quad q_\ell = \sup \|D_\theta h_\ell - D_\theta \widehat{h}_\ell\|_{\text{op}}.$$

With $e_0 = q_0 = 0$,

$$e_\ell \leq s_\ell w_\ell e_{\ell-1} + \epsilon_\ell.$$

Indeed, split the error into $\sigma(z) - \sigma(\widehat{z})$ and $\sigma(\widehat{z}) - p(\widehat{z})$. For derivatives, write

$$Z_\ell = D_\theta z_\ell, \quad \widehat{Z}_\ell = D_\theta \widehat{z}_\ell.$$

Because $D_\theta(W_\ell h + b_\ell)$ includes both $W_\ell D_\theta h$ and the direct increment $(\Delta W_\ell)h + \Delta b_\ell$,

$$\|Z_\ell - \widehat{Z}_\ell\|_{\text{op}} \leq w_\ell q_{\ell-1} + e_{\ell-1}.$$

If V_ℓ bounds $\|\widehat{Z}_\ell\|_{\text{op}}$, then

$$q_\ell \leq s_\ell (w_\ell q_{\ell-1} + e_{\ell-1}) + \left(h_\ell^{(2)} w_\ell e_{\ell-1} + \epsilon'_\ell \right) V_\ell.$$

This follows by adding and subtracting $D\sigma_\ell(z_\ell)\widehat{Z}_\ell$ in the hidden derivative difference. One usable recursive bound is

$$V_\ell \leq \sqrt{\widehat{H}_{\ell-1}^2 + 1} + w_\ell Q_{\ell-1}, \quad Q_\ell \leq (s_\ell + \epsilon'_\ell) V_\ell,$$

where $\widehat{H}_\ell$ bounds $\|\widehat{h}_\ell\|$ and Q_ℓ bounds $\|D_\theta \widehat{h}_\ell\|_{\text{op}}$. The output bounds are

$$\sup |f - \widehat{f}| \leq \|c\| e_L, \quad \sup \|\nabla_\theta f - \nabla_\theta \widehat{f}\| \leq e_L + \|c\| q_L.$$

The extra e_L comes from differentiating the trainable readout. Uniform bounds imply the corresponding L^2 bounds. Under controlled constants and a common small radius R , they give value order R^{k+1} and derivative order R^k . They do not prove that R is small or that the constants remain modest during training.

Appendix B. Implementation checks and companion code

The accompanying `local_tn_checks.py` constructs the quadratic cores from live PyTorch parameters and frozen SiLU derivative coefficients. Its default command runs the following finite numerical checks in float64:

1. Sequential polynomial evaluation equals tied-tree evaluation and explicit coefficient-tensor contraction.
2. Their gradients agree with respect to the original neural parameters.
3. The scalar polynomial in Section 4 has the stated coefficients.
4. Changing the centre of an exactly quadratic activation preserves the layer polynomial.
5. Exact polynomial translation agrees with direct evaluation of the old polynomial.

The optional `--demo` command runs a small staircase teacher, trains a depth-two SiLU student, and compares fixed-centre and example-centred quadratic forecasts. It is a smoke experiment, not the replication study of Section 10; its smaller configuration is printed and saved with the outputs. The script retains original parameterization and shared computation. It does not implement hierarchical tensor compression or establish population approximation bounds.

```
python local_tn_checks.py
python local_tn_checks.py --demo --output demo_results.json
```

The tests check implementation identities, not the research hypotheses. Numerical outcomes from the broader proposed sweeps, low-rank reduction, and causal intervention experiments are not claimed in this note.

References and project sources

- [P1] *Testing Tensor Networks as Toy Models of Deep Neural Network Training Dynamics: A staged teacher-student experimental program for hierarchical feature learning*. Project research plan, September 2026. Supplied as `DNN Training Plan.docx`; especially Sections 3-7 and 9-13.
- [P2] *Residual Tensor Networks and Soft Hierarchical Feature Learning: Exact liftings, population dynamics, and the role of input dependence*. Project research note, September 2026. Supplied as `Residual TTN Feature Learning.pdf`; especially Sections 3, 5, 7, and 8.
- [P3] *Toward a Realistic and Tractable Toy Model of Hierarchical Feature Learning in Pretraining: A residual tensor teacher-student program*. Project note, August 2026. Supplied as `Hierarchical Feature Learning Model.pdf`; especially Sections 3-4 and 9.
- [P4] *A Teacher-Student Toy Model of Hierarchical Feature Learning*. Project setup draft, 2026. Supplied as `draft.pdf`; especially Sections 4-5. Statements about its positive-control dynamics retain that draft’s specified parameterization and assumptions.
- [P5] J. Turnbull, B. Ottlik, and J. Simon. *A “feature ODE” describing the learning behavior of shallow MLPs on simple functions*. HiLD 2026 workshop manuscript. Supplied as `166_A_feature_ODE_describing_t.pdf`; Abstract and Sections 1-2.
- [R1] P. Morala, J. A. Cifuentes, R. E. Lillo, and I. Ucar. *NN2Poly: A Polynomial Representation for Deep Feed-Forward Artificial Neural Networks*. *IEEE Transactions on Neural Networks and Learning Systems* 36(1), 781-795, 2025. DOI: 10.1109/TNNLS.2023.3330328. Preprint: arXiv:2112.11397.
- [R2] Y. Bai, B. Krause, H. Wang, C. Xiong, and R. Socher. *Taylorized Training: Towards Better Approximation of Neural Network Training at Finite Width*. 2020. arXiv:2002.04010.
- [R3] J. Huang and H.-T. Yau. *Dynamics of Deep Neural Networks and Neural Tangent Hierarchy*. ICML, PMLR 119, 4542-4551, 2020.
- [R4] A. Meterez, P. A. Nair, D. Morwani, C. Pehlevan, S. Kakade, and A. Damian. *A Defense of the Quadratic Model*. Preprint, July 2026. arXiv:2607.21716. Cited for local optimization modelling, not for the tensor-network claims of this note.
- [R5] N. Cohen, O. Sharir, and A. Shashua. *On the Expressive Power of Deep Learning: A Tensor Analysis*. COLT, PMLR 49, 698-728, 2016.
- [R6] L. Grasedyck. *Hierarchical Singular Value Decomposition of Tensors*. *SIAM Journal on Matrix Analysis and Applications* 31(4), 2029-2054, 2010. DOI: 10.1137/090764189.
- [R7] C. Davis and W. M. Kahan. *The Rotation of Eigenvectors by a Perturbation. III*. *SIAM Journal on Numerical Analysis* 7(1), 1-46, 1970. DOI: 10.1137/0707001.